

Convex Layers

Anca Stanoev

anca.stanoev04@e-uvt.ro

West University of Timisoara

January, 2025

Abstract

The concept of convex layers in computational geometry refers to a decomposition of a set of points in the plane into nested convex hulls and it has applications in multiple fields such as statistics. This paper presents a comparative analysis of various algorithms for computing convex layers, including: Jarvis March, Graham's Scan, QuickHull, Andrew's (Monotone Chain) Algorithm and Chan's Algorithm. We start with elementary, baseline methods such as Jarvis March and gradually advance through more optimal ones such as Chan's Algorithm. The efficiency and behavior of the algorithms are evaluated in an experimental framework, analyzing both computational time and memory consumption across datasets of various distributions and dimensions, both static and dynamic. These include random uniform points, mostly collinear points, duplicates and replications of well-known mathematical constructs such as the Fibonacci Spiral, Koch Snowflake, and Sierpiński Triangle.

1 Introduction

The problem of Convex Layers is a well-studied theoretical aspect in Computation Geometry and it refers to a hierarchical computation of a series of nested convex polygons whose vertices include the points in the given set. The outer one is the convex hull of the points, and the rest are constructed in the same way recursively. The concept is not only an amazing object in the field of pure mathematics according to Joseph O'Rourke [3], but also has essential applications in the fields of pattern recognition, statistics, computer vision, robotics and computational topology. Moreover, in addition to the practical applications an interesting aspect of the problem is that it intuitively represents a geometric “equivalent” to sorting [1].

1.1 Motivation

Considering the diverse applications and computational approaches for convex layer algorithms, it is important to analyze their behavior across different data distributions to identify the most adaptable methods, in order to be able to predict an optimal approach. Additionally, in order to reconstruct the set of convex layers from scratch when deleting or inserting a point requires massive computational resources, especially when multiple layers are already available. Therefore, it is very important to use an algorithm that would not re-compute the layers from scratch upon insertion and deletion. This type of algorithm is known as a dynamic convex layers algorithm.

1.2 Paper Outline and Plan

In this paper, we outline five convex hull algorithms for a planar set of static points and their theoretical complexities. Additionally, we present an 'peeling' algorithm for a dynamic set of points that adapts the convex layers complete reconstruction. In chapters

2 and we formally present the problem of computing the convex layers of a planar set, in both a static and dynamic settings and address the theoretical aspects of each of the proposed algorithms. The purpose is to make a theoretical comparison of the behaviour of the algorithms across the following a dataset distributions: random uniform points, mostly collinear points, duplicate points, Fibonacci Spiral (replica) [4], Koch Snowflake (replica) [6] and the Sierpiński Triangle (replica) [2]. We also explore the benefits of implementing a dynamic peeling algorithm. In chapter 4 algorithms is tested , the algorithms are tested experimentally multiple data sizes. Finally, we provide a through comparison for each test case and propose future ideas for project improvement and expansion.

1.3 Limitations

Due to experimental constraints, we limit dataset sizes to 10,000 points for benchmarking and 1,000 points for animations to ensure real-time visualization. For geometric structures like the Sierpinski Triangle and Koch Snowflake, we approximate them with finite point sets that mimic their fractal forms, acknowledging their theoretical infinite nature that can not be implemented programmatically. Additionally, we explore edge cases such as mostly collinear points and duplicates to analyze how algorithms behave in non-typical or inefficient situations.

2 Formal Description of the problem and solution

Problem

Convex Layers (Static). Let $P \subset \mathbb{R}^2$ be a set of n points. Define the first layer $L_1 \subseteq P$ to be the set of points on the convex hull of P . Inductively, for $i > 1$,

$$L_i = \text{Conv}\left(P \setminus \bigcup_{j=1}^{i-1} L_j\right),$$

where $\text{Conv}(\cdot)$ denotes the convex hull operator. The *convex layers problem* (or onion decomposition) is to determine all L_i until no further hull can be formed.

Existing methods achieve from $\Theta(n^2)$ up to $\Theta(n \log n)$ time complexities. In particular, an $\Theta(n \log n)$ solution is optimal if each convex hull can be computed in $\Theta(n \log n)$ and updated efficiently.

Dynamic Setting. We also allow *insertions* and *deletions* of points into P . After each update, the layers $\{L_i\}$ must be recomputed or maintained. Approaches that are fully dynamic can be complex, with typical update times ranging from $O(n)$ to $O(n^2)$. Therefore, we are interested in providing a performance comparison across various point distributions and update operations, rating their efficiency, scalability, and adaptability in different computational scenarios.

2.1 Solution

To address the problem of comparison, we constructed a benchmarking algorithm that computes the data for each of the algorithms (running time in seconds, number of layers for each point distribution, memory complexity) across all datasets, for any given number of points. Its functionality is presented below. As for the peeling algorithm, we adapt a solution based on the work of Sadhu et al. [5], that reduces complexity per operation

2.1.1 Benchmarking Framework

We construct a benchmarking algorithm to evaluate the performance of various convex hull algorithms across multiple point distributions and input sizes. Let $\mathcal{A} = \{A_1, A_2, \dots, A_m\}$ represent the set of convex hull algorithms under consideration (e.g.,

Jarvis's March, Graham Scan, Andrew's Monotone Chain, Quickhull, Chan's Algorithm). For each point distribution $D \in \mathcal{D}$ and input size $n \in \mathcal{N}$, the benchmarking process involves:

1. **Point Generation:** Generate a point set $P \subset \mathbb{R}^2$ with $|P| = n$ according to distribution D .
2. **Algorithm Execution:** For each algorithm $A_i \in \mathcal{A}$, compute the convex layers $\{L_1^{(i)}, L_2^{(i)}, \dots, L_{k_i}^{(i)}\}$ of P , where $L_j^{(i)}$ denotes the j -th convex layer identified by A_i . Record the computation time $T_i(D, n)$ and the number of layers $k_i(D, n)$.
3. **Averaging Results:** Repeat the above steps r times for each (D, n) pair to obtain average computation times $\bar{T}_i(D, n)$ and average layer counts $\bar{k}_i(D, n)$.
4. **Performance Analysis:** Determine the most efficient algorithm $A^*(D, n)$ for each (D, n) by minimizing $\bar{T}_i(D, n)$:

$$A^*(D, n) = \arg \min_{A_i \in \mathcal{A}} \bar{T}_i(D, n).$$

2.1.2 Dynamic Convex Layers Algorithm

In dynamic contexts where points can be inserted or deleted from P , maintaining the convex layers $\{L_i\}$ efficiently is critical. Building on the algorithm proposed by Sadhu et al. [5], we implement a dynamic convex layers algorithm with the following properties:

Algorithmic Operations Let P_t denote the point set at time t . The dynamic algorithm supports two primary operations:

- **Insertion:** Upon inserting a new point $p \notin P_{t-1}$, the algorithm updates the convex layers by determining the minimal layer L_j such that $p \in \text{Conv}(P_t \setminus \bigcup_{i=1}^{j-1} L_i)$. The affected layers $L_j, L_{j+1}, \dots$ are recomputed. This operation incurs a time complexity of $O(n)$ up to $O(nh)$.
- **Deletion:** When deleting a point $p \in P_{t-1}$, the algorithm identifies the layer L_j containing p and removes it. Subsequently, layers $L_j, L_{j+1}, \dots$ are updated to maintain the correct hierarchical structure. This operation also requires $O(n)$ time, up to $O(nh)$ for worst case.

3 Theoretical Analysis

In this chapter, we analyze the theoretical aspects of the five algorithms for computing the onion decomposition of a static point set and for the implemented dynamic peeling algorithm. In the second part, we examine how different point distributions influence their performance and behavior.

Jarvis's March (Gift Wrapping)

Performance Analysis. Jarvis's March constructs the convex hull by iteratively selecting the next extreme point. Starting from the leftmost point p_0 , it selects p_{i+1} such that all points lie to the right of the line formed by p_i and p_{i+1} . This process is repeated until the hull is closed, resulting in a per-layer complexity of $O(nh_i)$, where h_i is the number of hull points in layer i .

Correctness. The algorithm ensures all extreme points are included by maintaining a counterclockwise order of selection. Each selected point p_{i+1} forms a boundary with p_i , guaranteeing the inclusion of all points on the convex hull without omissions or duplications.

Time and Space Complexity.

- *Time:* $O(nh)$, where n is the total number of points and h is the number of hull points. For k layers, the total complexity reaches:

$$\sum_{i=1}^k O\left(n - \sum_{j=1}^{i-1} h_j\right) \cdot h_i = O(n^2) \quad (\text{worst case})$$

- *Space:* $O(n)$, for storing the points and hull layers.

Advantages:

- Simple and intuitive implementation.
- Efficient for datasets with a small number of hull points.
- Naturally handles degenerate cases, such as collinear points.

Disadvantages:

- Inefficient for large datasets or hulls with many points.
- Time complexity degrades to $O(n^2)$ in the worst case where $h = O(n)$.
- Not suitable for real-time or large-scale applications.

Graham Scan

Performance Analysis. Graham Scan sorts the points based on their polar angles relative to the lowest point p_0 and constructs the convex hull by traversing the sorted list. The algorithm uses a stack to eliminate concave turns, resulting in a per-layer complexity of $O(n \log n)$ due to the sorting step.

Correctness. By sorting the points and ensuring that each turn during traversal is counterclockwise, Graham Scan guarantees the inclusion of all convex hull points without inward dents. The stack-based elimination of concave points maintains the hull's integrity.

Time and Space Complexity.

- *Time:* $O(n \log n)$, dominated by the initial sorting step.
- *Space:* $O(n)$, for storing the sorted points and the hull stack.

Advantages:

- More efficient than Jarvis's March for large datasets.
- Predictable performance with $O(n \log n)$ time complexity.

Disadvantages:

- Requires sorting, which can be computationally intensive.
- Less intuitive compared to Jarvis's March.

Andrew's Monotone Chain

Performance Analysis. Andrew's Monotone Chain constructs the convex hull by building the lower and upper hulls separately. It sorts the points lexicographically, then iterates through the sorted list to form each hull, ensuring counterclockwise turns. This also results in a per-layer complexity of $O(n \log n)$ due to the sorting step.

Correctness. By constructing the lower and upper hulls independently and enforcing counterclockwise turns, the algorithm accurately identifies all extreme points and assembles the complete convex hull without redundancies or omissions.

Time and Space Complexity.

- *Time:* $O(n \log n)$, mostly because of the initial sorting of points.
- *Space:* $O(n)$, for storing the sorted points and hull lists.

Advantages:

- Efficient with $O(n \log n)$ time complexity.
- Simpler implementation compared to some other algorithms.
- Effectively handles degenerate cases.

Disadvantages:

- Similar time complexity to Graham Scan, offering no significant performance gains.
- Requires careful handling of sorted point lists to maintain correctness.

Quickhull

Performance Analysis. Quickhull uses a divide-and-conquer strategy by recursively identifying hull segments. It selects extreme points, partitions the remaining points relative to the current hull edges, and constructs hull segments recursively. The average-case complexity is $O(n \log n)$, though it can degrade to $O(n^2)$ in the worst case where many points lie on the hull.

Correctness. By recursively partitioning the point set and eliminating points that lie inside the current hull segments, Quickhull ensures that all extreme points are included in the final convex hull. The divide-and-conquer approach guarantees coverage of all regions, which maintains the hull completeness.

Time and Space Complexity.

- *Time:* Average case $O(n \log n)$, but worst-case $O(n^2)$ when many points lie on the hull or are nearly collinear.
- *Space:* $O(n)$, for storing points and recursive stack space.

Advantages:

- Typically faster in practice compared to Jarvis's March.
- Simpler than some other divide-and-conquer algorithms.

Disadvantages:

- Worst-case time complexity of $O(n^2)$.
- Performance highly dependent on point distribution; may require robust pivoting to avoid inefficiencies.

Chan's Algorithm

Performance Analysis. Chan's Algorithm optimally combines divide-and-conquer with binary search techniques to achieve a time complexity of $O(n \log h)$, where h is the number of hull points. It partitions the point set into smaller subsets, computes convex hulls for each subset, and merges these hulls efficiently. This iterative process ensures that all hull points are identified without redundant computations.

Correctness. By bounding the number of hull points constantly and using efficient hull computations, Chan's Algorithm guarantees the identification of all convex hull points. The merging process ensures that no extreme points are omitted.

Time and Space Complexity.

- *Time:* $O(n \log h)$, which is optimal for convex hull computations.
- *Space:* $O(n)$, for storing points and intermediate hulls.

Advantages:

- Achieves optimal time complexity of $O(n \log h)$.
- Efficient for datasets where h is significantly smaller than n .
- Combines the strengths of divide-and-conquer and binary search techniques.

Disadvantages:

- More complex to implement compared to other convex hull algorithms.
- Overhead may not justify its use for small or moderately sized datasets.
- Requires careful management of subsets and merging processes to maintain efficiency.

Dynamic Peeling Algorithm

The dynamic convex layers algorithm presented in Chapter 2 achieves an overall time complexity of $O(n^2)$ in best and average cases for maintaining convex layers over a sequence of n insertions and deletions. Each update operation (insertion or deletion) is performed in linear time $O(n)$, if used efficiently.

Algorithmic Operations.

- **Insertion:** Upon inserting a new point $p \notin P_{t-1}$, the algorithm updates the convex layers by determining the minimal layer L_j such that $p \in \text{Conv}(P_t \setminus \bigcup_{i=1}^{j-1} L_i)$. Depending on what algorithm is used for recomposing the affected layers $L_j, L_{j+1}, \dots$, the operation incurs a time complexity of $O(n)$ up to $O(nh)$ where h is the number of points on the convex hull. The latter situation happens if we use a less efficient algorithm like Jarvis March.
- **Deletion:** When deleting a point $p \in P_{t-1}$, the algorithm identifies the layer L_j containing p and removes it. Subsequently, layers $L_j, L_{j+1}, \dots$ are updated to maintain the correct hierarchical structure using Jarvis's March. This operation also requires between $O(n)$ and $O(nh)$ time.

Time Complexity. Each insertion or deletion involves recomputing the convex hulls of the affected layers. Given that each operation may affect up to k layers, where k is the total number of layers, the overall time complexity remains $O(n^2)$.

Space Complexity. The algorithm maintains the entire set of points and their corresponding layers. The space required is $O(n)$, as each point is stored once, and layers are represented as subsets of the main point set. The auxiliary space for convex hull computations remains linear with respect to the number of points.

Point Distributions and Layer Growth

To analyze how the layers evolve, we consider several distributions for generating P . The computational cost depends on both k , the number of layers, and the hull sizes $\{h_i\}$.

3.1 Uniform Random

Points are sampled independently and uniformly over a rectangular region $[0, W] \times [0, H]$. Empirically, the outer hull L_1 tends to include a smaller fraction of the points, and subsequent hulls shrink relatively quickly. Typically,

$$k = O(\sqrt{n}) \quad (\text{on average}),$$

and each hull can be found in $O(n \log n)$. Thus, repeated peeling often remains practical, as the effective $\{n, h_i\}$ shrink at each iteration.

3.2 Mostly Collinear

A fraction of the points lie close to a line $y = ax + b$, and the rest are distributed randomly. Such near-degenerate configurations can force some hull algorithms (e.g., Quickhull without robust pivoting) toward worst-case $O(n^2)$. Even with methods like Graham Scan, the near-collinearity can cause large hulls:

$$|L_1| \approx \alpha n, \quad 0 < \alpha < 1.$$

Therefore, repeated hull extractions may not significantly reduce the effective input size, causing relatively large $\{h_i\}$ and a potentially higher total cost.

3.3 Duplicates

We replicate certain randomly chosen points, so multiple coordinates coincide. In most convex hull procedures, interior duplicates do not alter the hull boundary and thus do not affect $\{h_i\}$ significantly. However, care must be taken to handle degeneracies (e.g., multiple identical boundary points), which might result in bigger overhead due to repeated computations in algorithms like Jarvis March.

3.4 Fibonacci Spiral

We place point p_i at polar coordinates (r_i, θ_i) with

$$r_i = c\sqrt{i}, \quad \theta_i = i \cdot \alpha,$$

where $\alpha \approx 2.399963$ (the golden angle). Because this spiral expands radially and maintains a spread, the outer hull L_1 can be large (on the order of n). Subsequent hulls may also remain sizeable for multiple iterations if the “spiral arms” contain many extreme points. In the worst case, $\{h_i\}$ might not shrink rapidly, keeping the cost of repeated hull computations near $O(n \log n)$ each time and leading to a near-worst-case total complexity.

3.5 Sierpinski Triangle

Points lie within a fractal region formed by repeatedly taking midpoints toward the corners of an equilateral triangle. Empirically, a substantial fraction of points can cluster along triangular edges, causing each layer’s hull to remain large:

$$h_i \approx \beta n \quad \text{for multiple } i,$$

with $0 < \beta < 1$. Moreover, the fractal’s self-similar structure can yield many layers (k) before the remaining set becomes too small. Thus, if each peeling step removes only a moderate number of points, the total runtime can approach $k \cdot O(n \log n)$. Algorithms might approach near-worst-case behavior if the boundary remains dense layer after layer.

3.6 Koch Snowflake

Points lie within a fractal region formed by recursively adding smaller equilateral triangles to each side of an initial equilateral triangle, creating a Koch Snowflake-like structure. Empirically, a significant number of points cluster along the intricate fractal edges, resulting in large convex hulls for each layer:

$$h_i \approx \gamma n \quad \text{for multiple } i,$$

with $0 < \gamma < 1$. The fractal’s self-similar complexity leads to numerous layers (k) before the remaining set becomes negligible. As a result, each peeling step removes only a moderate number of points, causing the total runtime to approach $k \cdot O(n \log n)$. Convex hull algorithms may approach near-worst-case behavior due to the consistently large hulls and dense boundaries across layers.

4 Experiment

4.1 Experimental Setup and Results

In this section, we visually represent and compare the runtimes and complexities of each algorithm on each point distribution. Our methodology for testing the five convex-layer algorithms and our simple dynamic peeling approach is implemented in an internal <https://github.com/ancastanoev/Convex-Layers-application>

Based on the results acquired by the benchmarking algorithm, we present the algorithm comparison, grouped by each point distribution. To ensure consistency, we vary $n \in \{100, 1,000, 10,000, 100,000\}$ (whenever feasible in terms of time/memory), where n is the number of points in the planar set. All experiments were coded in Python 3.9 and ran on an Intel i7 CPU with 16 GB RAM, Windows 10 OS.

4.2 Results by Distribution

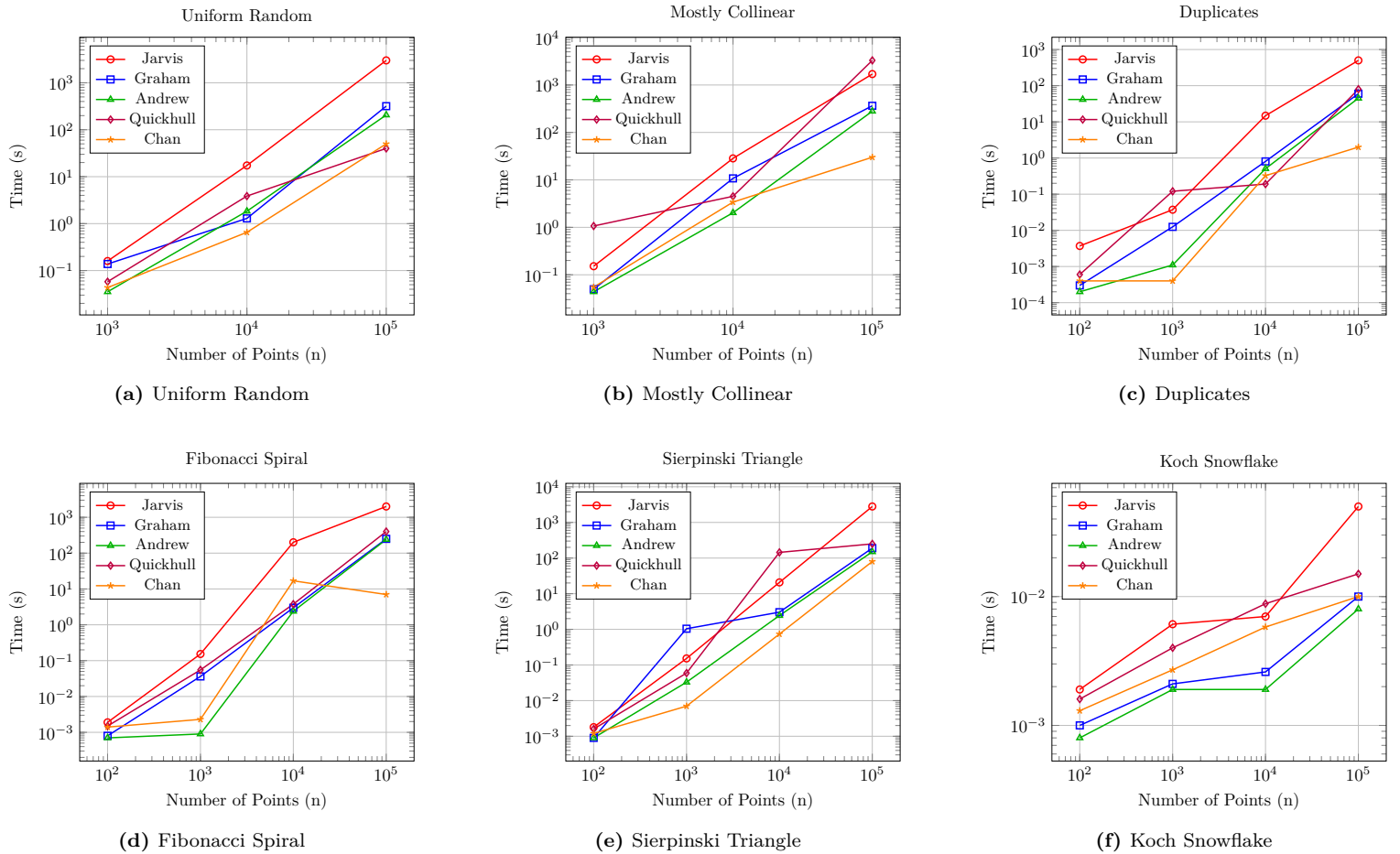


Figure 1: Runtime comparisons for all six distributions

1.Uniform Random

Algorithmic Implications. In uniformly random distributions (see Figure 1a), algorithms such as Graham Scan and Andrew’s Monotone Chain maintain their $O(n \log n)$ time complexity efficiently due to the rapid reduction in $\{h_i\}$ across layers. Quickhull and Jarvis’s March also perform well, benefiting from smaller hull sizes that prevent the algorithms from reaching their worst-case $O(n^2)$ complexities.

All methods perform relatively well for uniform data, with Quickhull and Chan slightly outperforming the others as n grows. The

layer count k increases roughly like $O(\sqrt{n})$ but does not massively affect the runtime for these methods.

Mostly Collinear

Algorithmic Implications. In near-collinear distributions (Figure 1b), Quickhull may degrade to $O(n^2)$ due to the hull size $|L_1| \approx n$. Graham Scan and Andrew’s Monotone Chain keep their $O(n \log n)$ per-hull complexity but have increased total computation cost. Jarvis March also performs more poorly, as it is inefficient when $|h|$ is large.

Duplicates

Algorithmic Implications. Duplicate points can adversely affect algorithms like Jarvis’s March and Quickhull by introducing redundant computations and requiring additional checks for identical points, potentially increasing runtime. On the other hand, Graham Scan and Andrew’s Monotone Chain handle duplicates during the sorting phase which lowers runtime and maintains $O(n \log n)$ efficiency.

Fibonacci Spiral

Algorithmic Implications. For the Fibonacci Spiral (Figure 1d), Quickhull and Jarvis’s March may experience inefficiencies due to large hull sizes $|h_i| \approx \Theta(n)$. Graham Scan and Andrew’s Monotone Chain maintain $O(n \log n)$ per hull extraction, but repeated peeling with many boundary points increases runtime per total. Chan’s Algorithm can avoid some overhead if it effectively leverages smaller merged subsets, yet still can be disadvantaged by partition overhead when $|L_1|$ is large.

Sierpiński Triangle

Algorithmic Implications. In Sierpiński Triangle distributions, Quickhull can approach $O(n^2)$ due to consistently large hull sizes. Graham Scan and Andrew’s Monotone Chain, while retaining $O(n \log n)$ per hull, face bigger total computation times if peeling repeatedly produces large hulls. Jarvis’s March also becomes inefficient, reaching near-quadratic runtime.

Koch Snowflake

Algorithmic Implications. In Koch Snowflake distributions (Figure 1f), fractal edges result in large hulls at each iteration. Quickhull and Jarvis’s March degrade toward $O(n^2)$. Graham Scan and Andrew’s Monotone Chain remain closer to $O(n \log n)$ per hull. Chan’s Algorithm can handle small- h scenarios optimally, but with large fractal boundaries, its theoretical advantage is sometimes not enough because of big overhead.

4.3 Dynamic Peeling Results

We also tested our *dynamic peel* procedure by inserting points one by one (random order) or deleting them from a static set. Depending on points distribution, complexity varies between $O(n)$ and $O(nh)$ per step. This is an advantage if we take leverage of more efficient algorithms like Chan, but can still result in large times for high n . For instance, at $n = 10^4$, each insertion might require several seconds in total over all increments.

More advanced data structures may reduce update costs to near $O(\log n)$ or $O(\log^2 n)$ for fully a dynamic hull, but that is beyond our current scope.

5 Result Analysis

We tested five algorithms on six distributions. Here is a succinct interpretation of the results:

- **Uniform Random (UR)**: Andrew’s and Graham are consistently near-optimal ($\mathcal{O}(n \log n)$); Quickhull also does well. - **Mostly Collinear (MC)**: Large hulls degrade Jarvis and Quickhull; Andrew or Graham handle them more robustly. - **Duplicates (DP)**: Sorting-based methods easily skip duplicates; Jarvis and Quickhull may see extra overhead. - **Fibonacci Spiral (FS)**: Big first hull slows Jarvis; Quickhull sometimes struggles, but Andrew/Graham remain stable. - **Sierpinski Triangle (ST) & Koch Snowflake (KS)**: Fractal edges create large hulls at multiple scales, penalizing Jarvis/Quickhull. Andrew and Graham remain safer.

In all cases, **Andrew’s Monotone Chain** and **Graham Scan** give the most consistent performance. **Chan’s Algorithm** can be optimal when the hull is much smaller than the full set. **Quickhull** is typically fast on random data but can degrade if many points lie on the hull. **Jarvis’s March** is only suitable for small sets or very small hulls. Our naive *dynamic* approach is correct for incremental updates but becomes slow beyond a few thousand points. However it maintains the $\mathcal{O}(n^2)$ complexity in most cases and exponentially outruns its static alternatives.

6 Related Work

Early work on layered convex hulls includes the paper by Chazelle [1], which introduced key ideas about repeatedly peeling off convex layers and explored their theoretical properties. More recent research, such as that by Wijeweera and Kodituwakku [7], studies how these layers can be maintained dynamically when points are inserted or deleted over time.

Our contribution builds on these foundations and extends the analysis by examining how different convex hull algorithms perform on a variety of dataset shapes, including fractal-like structures. We compare static algorithms side by side and look closely at issues like near-collinearity and point duplication.

7 Conclusions and Future Work

We have compared five algorithms (Jarvis March, Graham Scan, Andrew’s Monotone Chain, Quickhull, and Chan’s Algorithm) for creating convex layers, plus a dynamic method for adding or removing points. The focus was on analysing behaviour across edge-case datasets such as the aforementioned mathematical shapes and detect changes in behaviour.

Looking ahead, we plan to: Test bigger datasets to see how each algorithm handles more extreme sizes. Explore more efficient ways of updating layers dynamically, rather than recomputing from scratch. Study other shapes (like Hilbert curves) to see how each method handles even more unusual geometry.

References

- [1] B. Chazelle. “On the convex layers of a planar set.” In: *Ieee transactions on information theory* 31.4 (1985), pp. 509–517. DOI: 10.1109/TIT.1985.1057060.
- [2] Hans Meinhardt. “Triangles.” In: *The algorithmic beauty of sea shells*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 127–142.
- [3] J O’Rourke. *In computational geometry in ccambridge university press, cambridge, uk*. 1998.
- [4] TO Omotehinwa and SO Ramon. “Fibonacci numbers and golden ratio in mathematics and science.” In: *International journal of computer and information technology* 2.4 (2013), pp. 630–638.
- [5] S. Sadhu and N. Kumar. “Computing convex layers of a dynamic point set.” In: *International journal of computer theory and engineering* 7.6 (2015), pp. 495–498.
- [6] Eric W Weisstein. “Koch snowflake.” In: *Https://mathworld. wolfram. com/* (2008).

- [7] K. Wijeweera and S. Kodituwakku. “On the convex layers of a planar dynamic set of points.” In: *Ceylon journal of science* 47 (2018), p. 165. DOI: 10.4038/cjs.v47i2.7513.